Department of Computer Science — Data Structures and Algorithms

File Backup System Using Data Structure

A Project Report Submitted in Partial Fulfilment of the Requirements for the Course on Data Structures and Algorithms

Author	Role / Contribution
D. Sri Charan CDS/2025/1827 SCHOOL OF COMPUTING AND DATA SCIENCE SAI UNIVERSITY	System Design, Tree Implementation, DFS Algorithm
N.S.Mahammad Hujefa CDS/2025/1783 SCHOOL OF COMPUTING AND DATA SCIENCE SAI UNIVERSITY	Menu-Driven Interface, Testing & Documentation

ABSTRACT

The File Backup System using a Tree Data Structure is designed to manage multiple versions of files efficiently. In this system, each file version is represented as a node in a tree, where the root node represents the original file and child nodes represent subsequent versions. This structure allows users to perform operations such as adding, deleting, updating, searching, and displaying file versions. The system demonstrates how hierarchical data structures can be effectively used for version control and backup management. It provides a simple and efficient way to track file changes and maintain version history.

Keywords — Tree Data Structure, Depth First Search (DFS), Version Control, File Backup, Dynamic Memory Allocation, C Programming

I. INTRODUCTION

In modern computing, maintaining multiple versions of files is essential for backup, recovery, and tracking modifications. Traditional file systems do not efficiently manage version history, which can lead to data loss or confusion. This project introduces a File Backup System that uses a tree data structure to store file versions. Each node represents a version of a file, and the hierarchical structure reflects the relationship between versions. This approach mirrors version control systems used in professional software development environments such as Git.

The system is implemented in C using dynamic memory allocation and Depth First Search (DFS) for traversal. A menu-driven interface allows users to interact with the system through standard input/output operations. The primary goal is to illustrate how classic data structure principles can be applied to practical problems in file management and version tracking.

II. LITERATURE REVIEW

File version control and backup systems are widely used in computing. Systems such as Git and cloud storage services maintain file history and allow recovery of previous versions using advanced data structures and algorithms. Reema Thareja [1] and Cormen et al. [2] provide foundational treatments of tree structures and graph search algorithms that underpin such systems.

Tree data structures are commonly used to represent hierarchical relationships. DFS is a standard technique for traversing such structures and is well-documented for its use in pathfinding, cycle detection, and version graph traversal. This project applies these concepts in a simplified in-memory environment to demonstrate core version management principles.

III. OBJECTIVES

- Design a system for managing file versions using a tree structure.
- Implement operations: add, delete, update, search, and display.
- Demonstrate hierarchical data representation in C.
- Apply Depth First Search (DFS) for efficient node traversal.
- Provide a simple, working version control mechanism.

IV. SYSTEM DESIGN

A. Architecture

The system follows a root-to-leaf hierarchy. The root node represents the original file, and each child node represents a subsequent version. Each node in the tree stores the following fields:

```
typedef struct Node {
  int version;
  char fileName[100];
  struct Node* parent;
  struct Node* children[MAX];
  int childCount;
} Node;
```

B. Input / Process / Output

Input: User choices from the menu, file name, and parent version number. Process: Create or search nodes, add or delete versions, update file names, and traverse the tree. Output: Display the tree structure, show search results, and print confirmation messages.

V. IMPLEMENTATION DETAILS

The implementation is structured as a set of modular functions, each handling a specific operation on the tree. Dynamic memory allocation (malloc) is used to create nodes at runtime, allowing an arbitrary number of versions within the MAX_CHILDREN constraint.

A. createNode()

Allocates memory for a new node and initialises all fields including version number, file name, parent pointer, and child count to zero.

B. addVersion()

Accepts a parent version number and a new file name. Uses DFS (search()) to locate the parent node, then appends the new node to its children array if the child count limit is not exceeded.

C. search()

Performs a recursive Depth First Search starting from the root. Returns a pointer to the node whose version number matches the target, or NULL if not found. Time complexity: O(n) where n is the total number of nodes.

D. update()

Modifies the file name stored in an existing node. Calls search() to locate the target version before applying the update.

E. deleteVersion()

Restricts deletion to leaf nodes only (nodes with childCount == 0) to preserve version lineage. Locates the node via DFS, removes its pointer from the parent's children array, frees memory, and adjusts the parent's child count.

F. display()

Prints the full version tree to the console using indentation to reflect depth. Recursively visits each node in pre-order and indents output proportionally to the node's depth in the tree.

VI. METHODOLOGY

The development followed a structured, stepwise approach:

- Define the tree node structure using a C typedef struct.
- Instantiate the root node representing the original file version.
- Implement DFS-based search to locate parent nodes before insertion.
- Add child nodes for new versions, enforcing the MAX_CHILDREN limit.
- Restrict deletion to leaf nodes to prevent orphaned sub-trees.
- Display the tree with indentation proportional to node depth.
- Wrap all operations in a menu-driven loop for user interaction.

VII. RESULTS AND TESTING

The system was tested against the following scenarios using GCC on a Linux platform. All operations produced correct results consistent with the expected tree behaviour.

Table I summarises the tested operations and their outcomes.

VIII. TECHNOLOGY USED

- Programming Language: C (ISO C99)
- Compiler: GCC / Turbo C
- Concepts: Tree Data Structure, DFS, Dynamic Memory Allocation
- Platform: Any system with a C compiler

IX. SCOPE AND LIMITATIONS

The system supports version management of files including creation of multiple versions from a parent version and is suited for small-scale in-memory file tracking. However, it is limited to in-memory execution with no database or persistent storage integration, and the maximum number of children per node is bounded by a compile-time constant.

X. CONCLUSION

The File Backup System effectively demonstrates how a tree data structure can be used to manage file versions. It provides a clear understanding of hierarchical relationships and version tracking. The project is simple yet powerful in explaining core data structure concepts and version control basics. The DFS-based search ensures correct node discovery regardless of tree depth, and leaf-only deletion preserves the integrity of the version lineage.

XI. FUTURE WORK

- Integrate persistent file storage using a database or flat files.
- Remove the compile-time limit on the number of child nodes (unbounded branching).
- Implement a graphical user interface for improved usability.
- Add a version comparison (diff) feature.
- Implement undo/redo functionality using an auxiliary stack.
- Extend to a real-time, networked version control system.

XII. REFERENCES

[1] R. Thareja, Data Structures Using C, 2nd ed. Oxford University Press, 2014.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. MIT Press, 2009.

[3] GeeksforGeeks, "Tree Data Structure," [Online]. Available: https://www.geeksforgeeks.org/tree-data-structure/

[4] TutorialsPoint, "C Programming Tutorial," [Online]. Available: https://www.tutorialspoint.com/cprogramming/

APPENDIX A — TEST RESULTS (TABLE I)

Operation	Input	Expected Output	Result
createNode (root)	v1, original.c	Root node created	PASS
addVersion	parent=v1, v2_update.c	Child node added under v1	PASS
addVersion	parent=v2, v3_patch.c	Child node added under v2	PASS
search	version=v2	Pointer to v2 node returned	PASS
update	version=v2, revised.c	File name updated to revised.c	PASS
display	—	Tree printed with indentation	PASS
deleteVersion (leaf)	version=v3	v3 deleted, parent updated	PASS
deleteVersion (non-leaf)	version=v1	Deletion rejected (not leaf)	PASS
search (missing)	version=v99	NULL returned	PASS

APPENDIX B — FUNCTION COMPLEXITY SUMMARY

Function	Time Complexity	Space Complexity	Notes
createNode()	O(1)	O(1)	Single malloc call
addVersion()	O(n)	O(1)	DFS traversal to find parent
search()	O(n)	O(h)	h = height of tree (stack frames)
update()	O(n)	O(h)	Calls search() internally
deleteVersion()	O(n)	O(h)	Leaf-only, calls search()
display()	O(n)	O(h)	Pre-order DFS traversal